



apex-bancard-pos-plugin

POS Bancard – Cobro directo (cliente)

Oracle APEX Dynamic Action plugin: charge a physical Bancard POS terminal directly from the cashier's browser, with no application server in between. Framework-agnostic, drop into any APEX app.

Oracle APEX 20.2 / 22.x / 24.x

Dynamic Action plugin

MIT License

1 Why it exists, and how it works

A server-to-server integration with a Bancard POS terminal only works **if the application server can reach the terminal's local IP over the network**. In a lot of real deployments the app server and the terminal sit on different network segments, and that path never connects.

This plugin is the alternative path: the machine that *is* on the same LAN as the terminal is the **cashier's browser**, not the server. The plugin does a direct `fetch()` from the browser to the terminal's local IP/port — no backend in between.

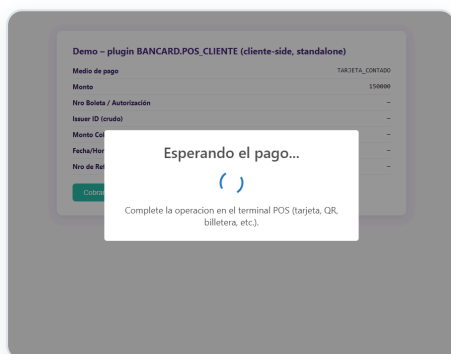
Coexists, doesn't replace. Built to work alongside an existing server-to-server integration, if you have one: use that path where the server can reach the POS, use this plugin where it can't. Both can live on the same page.

Execution flow

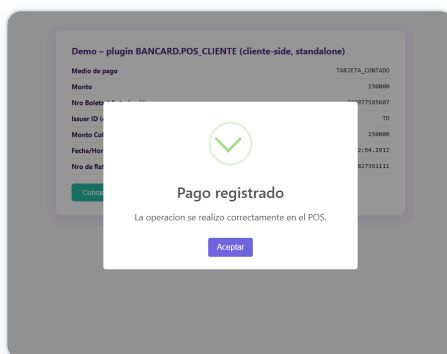
- 1 The cashier triggers the Dynamic Action (e.g. clicking a "Charge with POS" button).
- 2 The plugin's JavaScript does `POST /pos/eco` to the terminal (5s timeout) to confirm it's awake.
- 3 Depending on the configured **Medio de Pago**, it calls the matching terminal endpoint (90s timeout) — card, QR, e-wallet, etc.
- 4 Writes the terminal's **raw** result into the configured destination items.
- 5 All feedback —loading, success, error— shown with **SweetAlert2**, bundled with the plugin (no CDN dependency).

No round-trip to an application server in between: the entire Bancard protocol runs in the browser. The reference number (`facturaNro`) is generated with `Date.now()` , no database sequence involved.

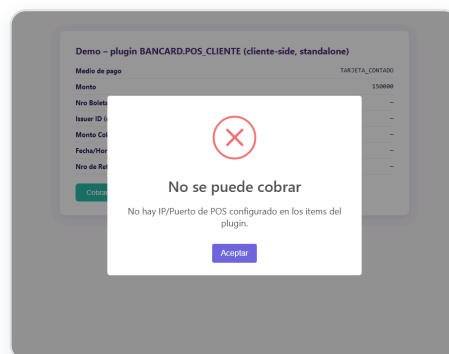
Feedback: SweetAlert2 modal, always



Loading — hand-built spinner in the modal's HTML (doesn't rely on `Swal.showLoading()` , see note below).
Never shows a button.



Success — modal with a green icon and an "Aceptar" button, not an auto-dismissing toast.



Error — same pattern for any failure: item validation, POS decline, timeout, or network/CORS issue.

Technical note — why the loader doesn't use `Swal.showLoading()` . That SweetAlert2 method works by converting the "Confirm" button into the spinner; if there's no visible button to convert (`showConfirmButton:false`), the spinner has nowhere to render. And if the swap isn't reverted cleanly before the next alert, the "Confirm" button can stay hidden forever. Instead, the plugin builds the spinner by hand in the modal's `html` (reusing the `.swal2-loader` CSS class already in the bundle) with `showConfirmButton:false` fixed: there's never a button during loading, and never anything to revert.

Mandatory browser requirement (real terminal only). The Bancard terminal speaks plain HTTP and doesn't send CORS headers. If your app is served over HTTPS, the browser blocks the `fetch()` unless, once per cashier machine: (1) `chrome://flags/#block-insecure-private-network-requests` is disabled, and (2) an "Allow CORS"-type extension is installed and enabled. Not needed against the bundled simulator (section 5): it already answers proper CORS headers.

2 Installation

Recommended: through the APEX UI

Shared Components → **Plug-ins** → **Create Plugin**, filling in the fields by hand:

- 1 **Identification:** Name `BANCARD.POS_CLIENTE`, Display Name "POS Bancard - Cobro directo (cliente)", Type *Dynamic Action*, Category *JavaScript*.
- 2 **Source** → **PL/SQL Code:** the `render` function (full code in `install/manual_ui_install.md`).
- 3 **Callbacks** → **Render Function Name:** `render`.
- 4 **Settings:** API Version 1, Supported UI Types Desktop, Substitute Attributes = Yes.
- 5 **Files:** upload `js/pos_bancard_cliente_bundle.js` (SweetAlert2 + plugin runtime, single file — two separate files in `JavaScript File URLs` render as one broken `<script>` tag).
- 6 **Custom Attributes:** the 11 attributes listed in section 3, all `Attribute Scope = Component`, `Type = Page Item`.

Full step-by-step with exact copy-paste field values: `install/manual_ui_install.md`.

Why not just run the SQL script? The `install/*.sql` scripts use `wwv_flow_api.create_plugin / create_plugin_attribute` with manually chosen IDs. In practice, a hand-picked ID can end up as a literal small number instead of a real APEX-generated ID (every other plugin in a real app has a 15-20 digit ID). A small ID can collide with something else in the instance's internal ID space, and since plugin metadata loads application-wide (not per-page), a single bad ID can break Page Designer for an entire application. Installing through the UI avoids this entirely — it's the same code path Oracle uses to generate every other plugin's ID.

Installing into another app / workspace

Once installed anywhere, the standard APEX way to move it: **Shared Components** → **Plug-ins** → open the plugin → **Export**, then in the target app **Shared Components** → **Plug-ins** → **Import File**. APEX resolves the target workspace/app/owner automatically and generates a real ID.

3 Integrating it into a charge/payment page

The pattern that works end-to-end has **three pieces** in the same Dynamic Action:

1. Resolve configuration

Native *Execute Server-side Code* action: PL/SQL with bind variables resolving IP/Port/Payment Method/Amount from your own app's logic, written into the plugin's configuration items.

2. The plugin's action

The Dynamic Action itself (*POS Bancard - Cobro directo (cliente)*), with the 11 attributes mapped to your page's items.

3. Map the result

A separate **Change** event on one of the plugin's destination items (not a chained action) that copies the raw result into your form's real fields.

Why piece 3 is a separate event, not a third chained action. The plugin's action runs asynchronously without telling the Dynamic Action engine to wait — as far as APEX is concerned, it "finishes" as soon as it's triggered, not when the POS responds. A chained action right after it would run immediately, before any result exists. The plugin does not suppress the **change** event on its destination items, so a separate Change-triggered action on one of them always fires once a result is actually in.

The plugin's 11 attributes

#	Attribute	Item content	Req.
1	Item: IP del POS	Terminal's local IP	Yes
2	Item: Puerto del POS	Terminal's port	Yes
3	Item: Medio de Pago	Payment method code (section 4)	Yes
4	Item: Monto	Amount, plain number (no thousands separator)	Yes
5	Item: Datos Adicionales (JSON)	JSON depending on the method — section 4	No
6	Item destino: Nro Boleta/Autorización	raw <code>nroBoleta</code> / <code>codigoAutorizacion</code>	Yes
7	Item destino: Issuer ID	raw <code>issuerId</code> (e.g. <code>VD</code> , <code>MC</code>) — unmapped	Yes
8	Item destino: Monto Cobrado	Charged amount, plain number	Yes
9	Item destino: Fecha/Hora de la Operación	Timestamp, ISO 8601	Yes
10	Item destino: Nro de Referencia	generated <code>facturaNro</code> (<code>Date.now()</code>)	Yes
11	Item destino: Resultado Completo (JSON)	full raw POS response	No

Piece 1 — example PL/SQL

Bind variables, not `apex_application.g_x01`. "Execute Server-side Code" reads/writes items with `:ITEM_NAME` directly, via "Items to Submit"/"Items to Return" — it doesn't wait for or print JSON with `sys.http.p`. That other pattern belongs to a page-level Ajax Callback **process** invoked from JavaScript via `apex.server.process(...)` — a different mechanism, different syntax. Mixing them means the action silently sets nothing.

```
declare
  l_ip_pos      my_terminals_table.ip_pos%type;
```

```

l_puerto_pos my_terminals_table.puerto_pos%type;
begin
  select t.ip_pos, t.puerto_pos into l_ip_pos, l_puerto_pos
    from my_terminals_table t
   where t.branch_id = :P_BRANCH_ID
        and t.register = :P_REGISTER
        and t.active   = 'Y';

  :P_POS_IP      := l_ip_pos;
  :P_POS_PUERTO := to_char(l_puerto_pos);
  :P_POS_MEDIO_PAGO := case :P_CARD_TYPE
                        when 'DEBIT' then 'TARJETA_DEBITO'
                        else 'TARJETA_CONTADO'
                        end;
  :P_POS_MONTO_PLANO := trim(replace(:P_AMOUNT, '.', ''));
exception
  when no_data_found then
    raise_application_error(-20001, 'No POS terminal configured for this register.');
```

```
end;
```

Piece 3 — example result mapping

```

function formatThousands(n) {
  n = Math.round(Number(n) || 0);
  return n.toString().replace(/\B(?=(\d{3})+(?!\d))/g, '.');
}
apex.item("MY_RECEIPT_NUMBER_ITEM").setValue(
  formatThousands(apex.item("P_POS_NRO_BOLETA").getValue())
);
apex.item("MY_AMOUNT_ITEM").setValue(
  formatThousands(apex.item("P_POS_MONTO_COBRADO").getValue())
);
```

4 Payment methods and downstream mapping

Supported payment methods

Code (Item: Medio de Pago)	Bancard endpoint(s)	Datos Adicionales (JSON)
TARJETA_CONTADO	venta-ux → descuento	—
TARJETA_CUOTAS	venta-ux (installments) → descuento	{"cuotas":N,"plan":N}
TARJETA_DEBITO	venta/debito → descuento	—
TARJETA_CREDITO	venta/credito → descuento	{"cuotas":N,"plan":N}
QR	venta-qr	{"montoVuelto":N,"promotions":[...]} (optional)
QR_PIX	venta-qr-pix	{"pix_payer_cpf":"...","pix_payer_phone":"..."}
EXTRACCION_QR	extraccion-qr	—
CANJE	venta-canje	—
CANJE_QR	venta-canje-qr	—
BILLETERA	venta-billetera	{"billetera":"ZIM","cuenta":"123456"}

`anulacion` / `consulta-anulacion` are deliberately out of scope: post-sale management operations, not payment methods.

Downstream mapping (each app's own responsibility)

Each company keeps its own card-brand catalog (`issuerId` → `internal brand id`). The plugin doesn't know it; the mapping happens in the page, typically in the same "Change" event from piece 3:

```
var myBrand = MY_BRAND_CATALOG[ apex.item('P_POS_ISSUER_ID').getValue() ] || 99;
apex.item('MY_CARD_BRAND_ITEM').setValue(myBrand);

// Same applies to date formatting: convert P_POS_FECHA (ISO 8601)
// to whatever format your save process expects.
```

This is intentionally each app's responsibility, not the plugin's — it's what lets the same, unmodified plugin be installed in any application, with its own brand catalog and its own save format.

5 Testing without a physical terminal

`simulator/pos_simulator.py` implements all 13 real Bancard v1.5.0 endpoints, runs on the same machine or over the LAN, no external dependencies:

```
python pos_simulator.py --port 3000
```

Simulating realism: delay and random outcomes

Flag	What it does
<code>--delay-cliente N</code> or <code>N-M</code>	Simulates how long the customer takes to complete the transaction. Fixed value or random range (different every sale). Applies to everything except <code>/pos/eco</code> .
<code>--random</code>	Instead of always approving, randomly rejects sales (insufficient funds, expired/blocked card, etc.), times out, or fails <code>/pos/eco</code> itself.
<code>--fail-rate</code>	Rejection probability (0-1). Default 0.15.
<code>--timeout-rate</code>	Timeout probability (0-1). Default 0.03.
<code>--eco-fail-rate</code>	Probability the <code>/pos/eco</code> check fails. Default 0.05.
<code>--interactive</code>	Approve/reject/timeout each sale from the console instead, to reproduce a specific case.

```
python pos_simulator.py --port 3000 --delay-cliente 5-10 --random
```

Mind the gap between simulated delay and the plugin's timeout. The plugin's operation timeout is 90 seconds. If `--delay-cliente`'s max is close to or above that, the browser can report `Failed to fetch` instead of completing the test — keep the delay range comfortably below the timeout.

Full endpoint/flag reference: `simulator/README.md`. To try the plugin alone, no APEX app needed: `js/demo.html`.

6 Troubleshooting

Symptom	Likely cause
<code>Failed to fetch</code> against a real terminal	Missing <code>chrome://flags</code> + "Allow CORS" extension setup on that machine.
<code>Failed to fetch</code> against the simulator	<code>--delay-cliente</code> too close to the plugin's 90s timeout — lower the delay range.
"POS did not respond in time"	Terminal off, wrong IP/port, disconnected cable, or (against the simulator) <code>--random</code> rolled a timeout.
Configuration items arrive empty at the plugin's action	"Execute Server-side Code" (piece 1) written with <code>g_x01 / sys.http.p</code> instead of bind variables — see section 3. No error thrown, just sets nothing.
Plugin action chained after another async action never gets its attributes	Move the result mapping to a separate "Change" event (piece 3), not a chained action — see section 3.
Charge button doesn't show/hide live as the form changes	A server-side button condition only evaluates at page render — add your own show/hide Dynamic Action bound to the relevant items.
Page Designer breaks app-wide (can't open any Dynamic Action, any page)	A plugin installed with a manually-typed small ID via <code>wwv_flow_api</code> — see the danger box in section 2. Uninstall with <code>WWV_FLOW_API.REMOVE_PLUGIN</code> and reinstall through the UI.

7 APEX version compatibility

Built and tested on APEX 20.2. The API this plugin's `render` function uses is stable and documented from APEX 20.1 onward:

- **The `apex_plugin.t_dynamic_action` / `t_dynamic_action_render_result` API is stable and documented.** Comparing the official definitions for 20.1 against 24.2, the 15 `attribute_01` .. `attribute_15` fields this plugin uses didn't change; 24.2 only adds new fields at the end this plugin doesn't need.
- **SweetAlert2 ships bundled** as the plugin's own file, not loaded from an external CDN — a strict Content Security Policy `script-src` would block a `<script>` pointing at an external CDN domain not explicitly allow-listed.
- **APEX 24.1 deprecated "Substitute Attribute Values"** for region plugins; doesn't affect this plugin, which uses the simpler, stable Dynamic Action pattern (`p_render_function` only).

Installing through the UI (section 2, recommended path), version compatibility is handled by APEX itself — doesn't depend on any hand-typed `p_release`. The `install/*.sql` scripts remain in the repo mainly as a readable reference for the plugin's source code.